Free Monad

ree · Bank-transfer DSL with Map interpreter + logging optimizer · Build with Waqa

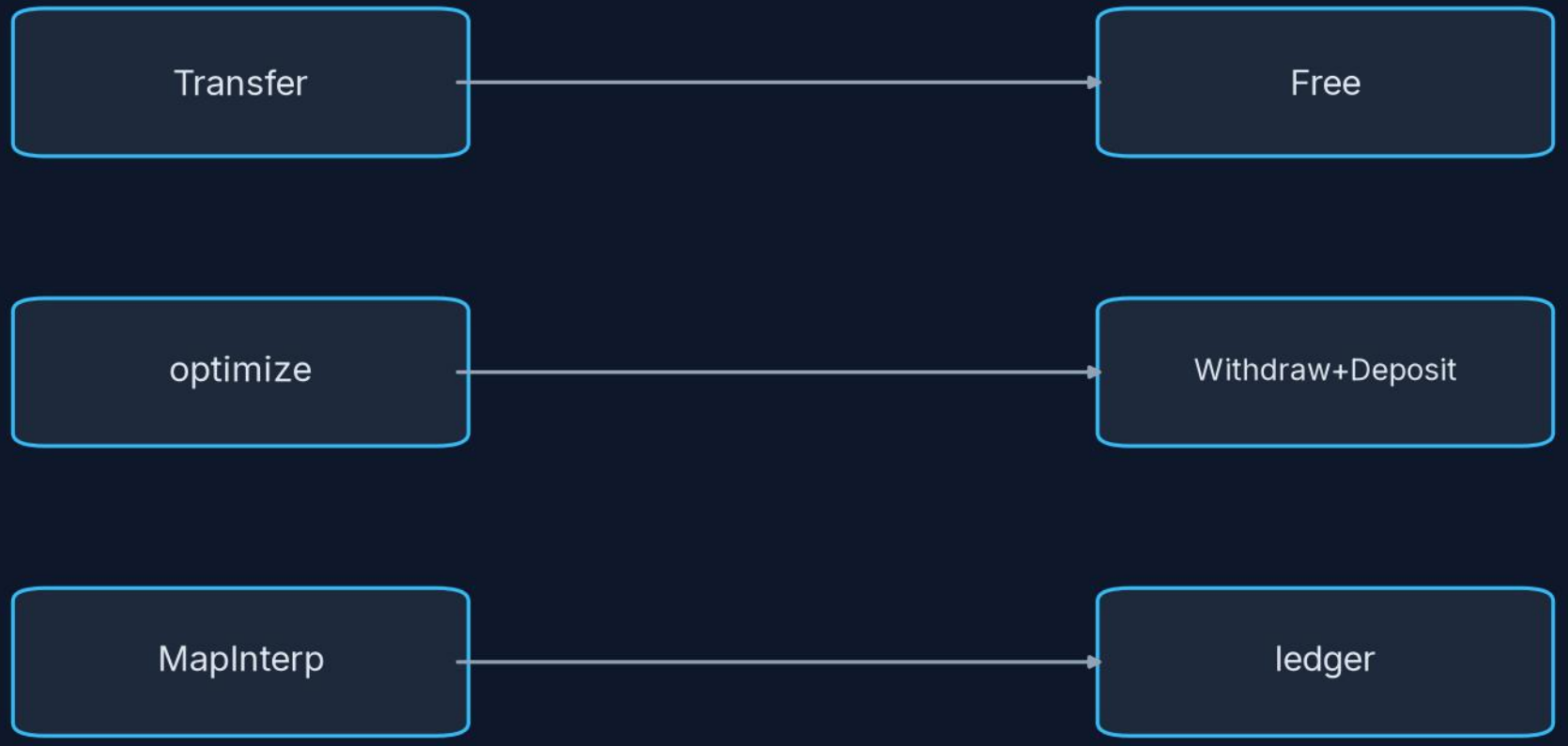
Today's Hook

- Problem: Bank-transfer DSL with Map interpreter + logging optimizer
- Pattern: Free — earned from the pain, not trivia.
- Runnable main on JDK 25 with captured stdout.
- Naive → solution → failure paths → trade-offs.

The Problem

- Bank-transfer DSL with Map interpreter + logging optimizer
- Naive design shows the cost clearly.
- Pattern isolates the force behind a clean boundary.
- Tests cover happy path and injected failures.

Structure



Free: Bank-transfer DSL with Map interpreter + logging o

Real-World Scenario

- Running example: Bank-transfer DSL with Map interpreter + logging optimizer
- Same names in naive code, refactor, and Demo output.
- No shared StreamForge framing unless it fits.
- Product language first; pattern vocabulary second.

Naive Code

```java
package com.buildwithwaqar.patterns.functional.free.problem;

import java.util.LinkedHashMap;
import java.util.Map;

/** Naive: transfer logic inlined; no program to optimize or log separately. */
public final class NaiveBank {
    private final Map<String, Integer> ledger = new LinkedHashMap<>();
    public void seed(String acct, int cents) { ledger.put(acct, cents); }
    public int transfer(String from, String to, int cents) {
        int bal = ledger.getOrDefault(from, 0);
        if (bal < cents) throw new IllegalStateException("insufficient:" + from);
        ledger.put(from, bal - cents);
        ledger.put(to, ledger.getOrDefault(to, 0) + cents);
        return ledger.get(to);
    }
    public Map<String, Integer> ledger() { return Map.copyOf(ledger); }
}
```

Why Naive Hurts

- Call sites couple to the wrong shape or leak on partial failure.
- Parameter drilling / duplicated envelopes / missing rollback.
- Hard to unit-test without the real dependency.
- New variants require shotgun edits.

Solution

```java
package com.buildwithwaqar.patterns.functional.free.problem;

import java.util.ArrayList;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
import java.util.function.Function;

/** Free-style bank DSL: Balance / Withdraw / Deposit / Transfer as an AST of instructions. */
public final class BankDsl {
    private BankDsl() {}

    public sealed interface Instr permits Balance, Withdraw, Deposit, Transfer {}
    public record Balance(String account) implements Instr {}
    public record Withdraw(String account, int cents) implements Instr {}
    public record Deposit(String account, int cents) implements Instr {}
    public record Transfer(String from, String to, int cents) implements Instr {}

    public abstract static class Free<A> {
        private Free() {}
        public abstract <B> Free<B> flatMap(Function<? super A, Free<B>> f);
        public final <B> Free<B> map(Function<? super A, ? extends B> f) {
            return flatMap(a -> pure(f.apply(a)));
        }
```

Solution (2)

```
    public static <A> Free<A> pure(A a) { return new Pure<>(a); }
    public static Free<Integer> balance(String acct) { return new Suspend<>(new Balance(acct)); }
    public static Free<Void> withdraw(String acct, int cents) { return new Suspend<>(new Withdraw(acct, cents)); }
    public static Free<Void> deposit(String acct, int cents) { return new Suspend<>(new Deposit(acct, cents)); }
    public static Free<Void> transfer(String from, String to, int cents) {
        return new Suspend<>(new Transfer(from, to, cents));
    }
}

private static final class Pure<A> extends Free<A> {
    final A value;
    Pure(A value) { this.value = value; }
    public <B> Free<B> flatMap(Function<? super A, Free<B>> f) { return f.apply(value); }
}
private static final class Suspend<A> extends Free<A> {
    final Instr instr;
    Suspend(Instr instr) { this.instr = instr; }
    public <B> Free<B> flatMap(Function<? super A, Free<B>> f) { return new Bind<>(this, f); }
}
private static final class Bind<A, B> extends Free<B> {
    final Free<A> prev;
    final Function<? super A, Free<B>> f;
    Bind(Free<A> prev, Function<? super A, Free<B>> f) { this.prev = prev; this.f = f; }
    public <C> Free<C> flatMap(Function<? super B, Free<C>> g) {
        return new Bind<>(prev, a -> f.apply(a).flatMap(g));
    }
```

Solution (3)

```
    }

    /** Optimize Transfer into Withdraw+Deposit for the Map interpreter.
```

Runnable main

```java
package com.buildwithwaqar.patterns.functional.free.problem;

import java.util.ArrayList;
import java.util.LinkedHashMap;
import java.util.List;

/** Problem: bank-transfer DSL — Map interpreter + logging + Transfer optimizer. */
public final class ProblemDemo {
    public static void main(String[] args) {
        System.out.println("=== Free: bank-transfer DSL ===");
        System.out.println("--- naive ---");
        var naive = new NaiveBank();
        naive.seed("payroll", 10_000);
        naive.seed("alice", 0);
        System.out.println("toBal=" + naive.transfer("payroll", "alice", 2500));
        System.out.println("ledger=" + naive.ledger());

        System.out.println("--- Free program + Map interpreter ---");
        var ledger = new LinkedHashMap<String, Integer>();
        ledger.put("payroll", 10_000);
        ledger.put("alice", 0);
        var prog = BankDsl.paydayBonus("payroll", "alice", 2500);
        int bal = BankDsl.interpretMap(prog, ledger);
        System.out.println("toBal=" + bal + " ledger=" + ledger);
```

Runnable main (2)

```
System.out.println("--- logging interpreter ---");
var ledger2 = new LinkedHashMap<String, Integer>();
ledger2.put("payroll", 10_000);
ledger2.put("bob", 0);
List<String> log = new ArrayList<>();
BankDsl.interpretMap(BankDsl.Free.transfer("payroll", "bob", 100), ledger2, log);
System.out.println("log=" + log);

System.out.println("--- optimized Transfer → Withdraw+Deposit ---");
var opt = BankDsl.optimize(BankDsl.Free.transfer("payroll", "bob", 50));
var ledger3 = new LinkedHashMap<String, Integer>();
ledger3.put("payroll", 10_000);
ledger3.put("bob", 100);
List<String> log3 = new ArrayList<>();
BankDsl.interpretMap(opt, ledger3, log3);
System.out.println("optLog=" + log3);
System.out.println("ledger=" + ledger3);

System.out.println("--- failure: insufficient funds ---");
try {
    var bad = new LinkedHashMap<String, Integer>();
    bad.put("empty", 0);
    BankDsl.interpretMap(BankDsl.Free.withdraw("empty", 1), bad);
} catch (IllegalStateException e) {
    System.out.println("caught=" + e.getMessage());
}
```

Runnable main (3)

```
        }
    }
```

Walkthrough Output

```
=== Free: bank-transfer DSL ===
--- naive ---
toBal=2500
ledger={payroll=7500, alice=2500}
--- Free program + Map interpreter ---
toBal=2500 ledger={payroll=7500, alice=2500}
--- logging interpreter ---
log=[Transfer[from=payroll, to=bob, cents=100], Withdraw[account=payroll, cents=100], Deposit[account=bob, cents=100]]
--- optimized Transfer → Withdraw+Deposit ---
optLog=[Withdraw[account=payroll, cents=50], Deposit[account=bob, cents=50]]
ledger={payroll=9950, bob=150}
--- failure: insufficient funds ---
caught=insufficient:empty
```

Step-by-Step Trace

- Each stdout line maps to a stage above.
- Re-run the main class locally on JDK 25.
- Failure-path lines prove compensations / rejects / missing files.
- Pause and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction
// records + sealed + switch
// inject Env / collaborators at the edge
// virtual threads for blocking fan-out
```

Trade-offs

- Clarity vs ceremony — refuse five types for two lines.
- Teaching demos omit production metrics/idempotency keys.
- Prefer simpler language features when they erase the force.
- Know the twin pattern often confused with Free.

Related Patterns

- Contrast Free with its neighbor pattern.
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Problem → naive → Free → Demo stdout.
- Failure paths are part of the lesson.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run the main.